ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

Бие даалт 1

Үйлдлийн систем (S.HDM101)

2026-2027 оны хичээлийн жилийн намар

Семинарын цаг: 5-1 пар

Хичээл заасан багш: ..... Г. Ганбат/ F.CS10/

Хийсэн оюутан:..... Н. Ням-Од / B222270805/

..... А. Гэрэл / B241940001/

..... А. Номин/ B242270088/

..... Э. Мягмаршагай/ B241910001/

..... О. Хүслэн/ B241910020/

..... Ч. Түмэншагай/ B160910830/

Агуулга

Хураангуй 3

1. Удиртгал 3

2. Сурах бичгийн шаардлага 3

3. Онолын үндэс 4

4. Системийн зохиомж 5

5. Хэрэгжилт 5

6. Туршилт ба UI дүрслэл 5

7. Туршилтын арга 8

8. Үр дүн 8

9. Хэлэлцүүлэг 11

10. Дүгнэлт 11

Ашигласан материал 11

Хураангуй

Асуудал: kernel-д ашиглагддаг холбоост жагсаалт, дараалал, стек, bitmap, мод зэрэг өгөгдлийн бүтцийн ажиллагааг ойлгох.

Шийдэл: list_head-ийн next/prev семантикийг C-ээр дахин хэрэгжүүлж, нэг командын хэл бүхий CLI ба түүнтэй ижил хөдөлгүүртэй interactive UI (Kernel structure graph) бүтээв; бодит kernel module-ийг мөн бичсэн.

Номын 1.9-ийн бүтэц бүр: жагсаалт, дараалал, стек, энгийн ба тэнцвэржсэн (AVL) мод, хэш хүснэгт (collision-той), bitmap-д нэмэх, устгах, хайх үйлдлийг хэрэгжүүлж графаар харуулсан. **Нэмэлт:** бэлэн дараалал (ready queue), PCB, FCFS ба Round Robin scheduler-ыг мөн хэрэгжүүлж хуваарилалтын хүснэгтэд заасан «Processes» сэдэвтэй холбосон.

Туршилт: 6 жишээ скрипт + 5000 давталттай stress тест ASan/UBSan-тай давав; C ба UI-ийн хөдөлгүүр 6 скрипт болон 40 000 санамсаргүй командад (8 seed × 5000, тэр дотор тэнцвэржсэн мод) яг ижил гаралт өгсөн; AVL-ийн өндрийн хязгаарыг тусад нь шалгасан; kernel module Linux 6.11.0-17 дээр compile хийгдэж, insmod/rmmod-оор ачаалагдан dmesg-ээр ажиллагаа нь баталгаажсан.

Гол үр дүн: оролт, огноо, хязгаарыг шалгадаг, хэрэглэгч өөр өгөгдлөөр дахин туршиж болох хэрэгсэл.

1. Удиртгал

Үйлдлийн системийн kernel нь процессын жагсаалт, ready queue, bitmap (аль нэг нөөц чөлөөтэй эсэх), модон бүтэц (scheduler, файл систем) зэргийг өгөгдлийн бүтцээр удирддаг. Төслийн зорилго нь эдгээрийн үндсэн үйлдлийг хэрэгжүүлж, нүдэнд харагдахуйц болгох. Хамрах хүрээ: хэрэглэгчийн орчны загвар + нэг kernel module; бодит scheduler-д холбохгүй.

2. Сурах бичгийн шаардлага

Хуваарилагдсан сэдэв: сурах бичгийн **1.9 Kernel Data Structures** (Operating System Concepts, 1-р бүлэг).

Хүрэх үр дүн: «сурах бичигт заасан kernel data structure-ийн хэрэглээг хэрэгжүүлж, элемент нэмэх, устгах, хайх үйлдлийн дараах бүтцийг графаар харуулна». Хуваарилалтын хүснэгт: үзүүлэлт «Kernel structure graph», холбогдох сэдэв «Processes». Доорх хүснэгт нь 1.9-ийн агуулга бүрийг хэрэгжилт ба тестэд харгалзуулсан.

1.9 агуулга	Хэрэгжилт	Нэмэх / устгах / хайх	Баталгаа
1.9.1 Холбоост жагсаалт (singly, doubly, circular; O(n) хайлт)	Kernel-ийн list_head шиг **давхар холбоост дугуй** жагсаалт (struct birthday)	add_head, add_tail, del, find	textbook, search, UI «Галт тэрэг»
1.9.1 Stack (LIFO, push/pop)	list_head дээр, дээд талаас нь	push, pop, find	all_structures, search
1.9.1 Queue (FIFO)	list_head дээр, төгсгөлд нэмж эхнээс авна	push, pop, find	all_structures, search
1.9.2 Binary search tree (хамгийн муу O(n))	Энгийн BST	insert, delete, find	search: 1–5 дараалан өгөхөд өндөр 5
1.9.2 Тэнцвэржсэн мод (O(lg n))	**AVL** (tree mode avl)	insert, delete, find	search, avl_check: 1–7 → өндөр 3
1.9.3 Хэш функц ба hash map, collision	Хэш функц = тэмдэгтийн кодын нийлбэр mod n; collision-д жагсаалт залгана	put, del, get	search: bob, ab, ba нэг нүдэнд
1.9.4 Bitmap (n item-ийн төлөв)	n bit бүхий самбар	set, clear, test	all_structures, search
Linux kernel хайрцаг: list.h, kfifo, rbtree.h	kernel module-д list_head ба kfifo ашигласан; rbtree.h **ашиглаагүй** (AVL-аар загварчилсан)	insmod / rmmod	Зураг 5

Харуулах шаардлага: бүтэц бүрийн нэмэх, устгах, хайх үйлдлийн дараах төлөвийг UI дээр граф (вагон, дараалал, овоолго, нүд, мод, bucket) хэлбэрээр шууд зурдаг.

3. Онолын үндэс

list_head – intrusive, давхар холбоост, дугуй жагсаалт. Node өөрөө өгөгдлийн struct дотор list_head талбар агуулж, container_of макроор эзэн struct-ийг сэргээнэ. Нэмэх/устгах нь O(1), хайх нь O(n).

Queue (FIFO) – tail-д нэмж, head-ээс авна.

Stack (LIFO) – head-д нэмж, head-ээс авна.

Bitmap – 1 bit = 1 нөөц; set/clear/test O(1), 8 бит Boolean-аас 8 дахин бага зай.

Binary search tree – зүүн < үндэс < баруун; дундаж O(lg n), харин эрэмбэлсэн оролтод нэг шугам болж хамгийн муу O(n) – тиймээс **тэнцвэржсэн мод** (n элементэд хамгийн ихдээ ойролцоогоор lg n түвшин) хэрэглэдэг; Linux red-black мод ашигладаг, бид AVL-аар загварчилсан.

Hash map – хэш функц түлхүүрийг хүснэгтийн индекс болгож, боломжийн үед O(1) хайлт өгнө; хоёр түлхүүр нэг индекст таарвал (collision) тэр нүдэнд жагсаалт залгана, collision олон

болох тусам хайлт O(n) руу ойртоно. Нийтлэг алдаа: *list_del*-ийн дараа pointer-ыг ашиглах, node-ыг чөлөөлөхгүй үлдээх (leak), iterate хийж байхдаа устгах (_safe хувилбар хэрэгтэй).

4. Системийн зохиомж

Компонент: (a) **kds.c** – командын интерпретатор + 5 бүтэц; (b) **index.html** – ижил команд, мөн ижил хөдөлгүүр, SVG график; (c) **kds_module.c** – kernel орчны хувилбар. Өгөгдлийн урсгал: хэрэглэгчийн команд → интерпретатор → бүтцийн төлөв шинэчлэх → мессеж (OK/ERR) → (UI дээр) төлөвөөс график зурах. График нь тусдаа өгөгдөл биш, шууд бүтцийн төлөвөөс зурагддаг тул бодит ажиллагаатай үргэлж нийцнэ.

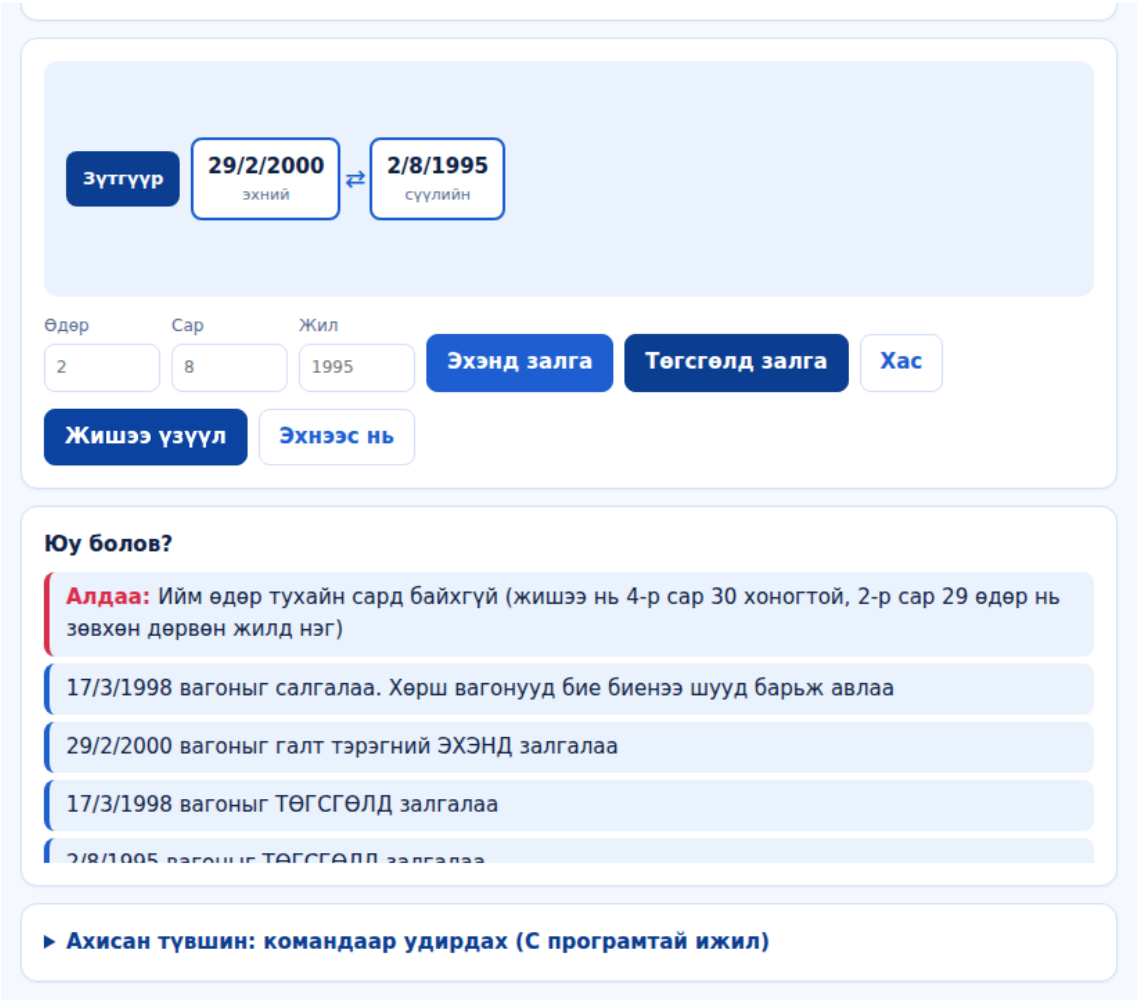
5. Хэрэгжилт

Хэл: C (gcc, -fsanitize=address,undefined), JavaScript (SVG), kernel C. Гол шийдлүүд: (1) list.h-ийн __list_add, list_add, list_add_tail, list_del, container_of-ийг өөрөө бичсэн; (2) queue ба stack нь ижил list_head-ийг хуваалцаж зөвхөн нэмэх талаараа ялгарна; (3) огноо шалгалт нь хаврын жилийг (4-т хуваагдах, 100-д биш, эсвэл 400) тооцно; (4) модны устгал нь хоёр хүүтэй үед баруун дэд модны хамгийн бага утгаар солино; (5) kernel module-д list_for_each_entry_safe, kmalloc алдаа гарвал цэвэрлэх замтай; (6) **Процесс ба scheduler:** процесс бүр struct pcb {pid, name, rem, slice, list_head} бөгөөд бэлэн дараалал нь дахин list_head-ийн FIFO (list_add_tail – нэмэх, эхнийхийг list_del – CPU-д өгөх). sched tick бүрд CPU 1 тик ажиллана; FCFS-д дуустал, Round Robin-д квант дуусвал процессыг дарааллын төгсгөлд буцаана (preempt). Хүлээлтийн хугацаа = нийт үргэлжилсэн – ажлын хугацаа.

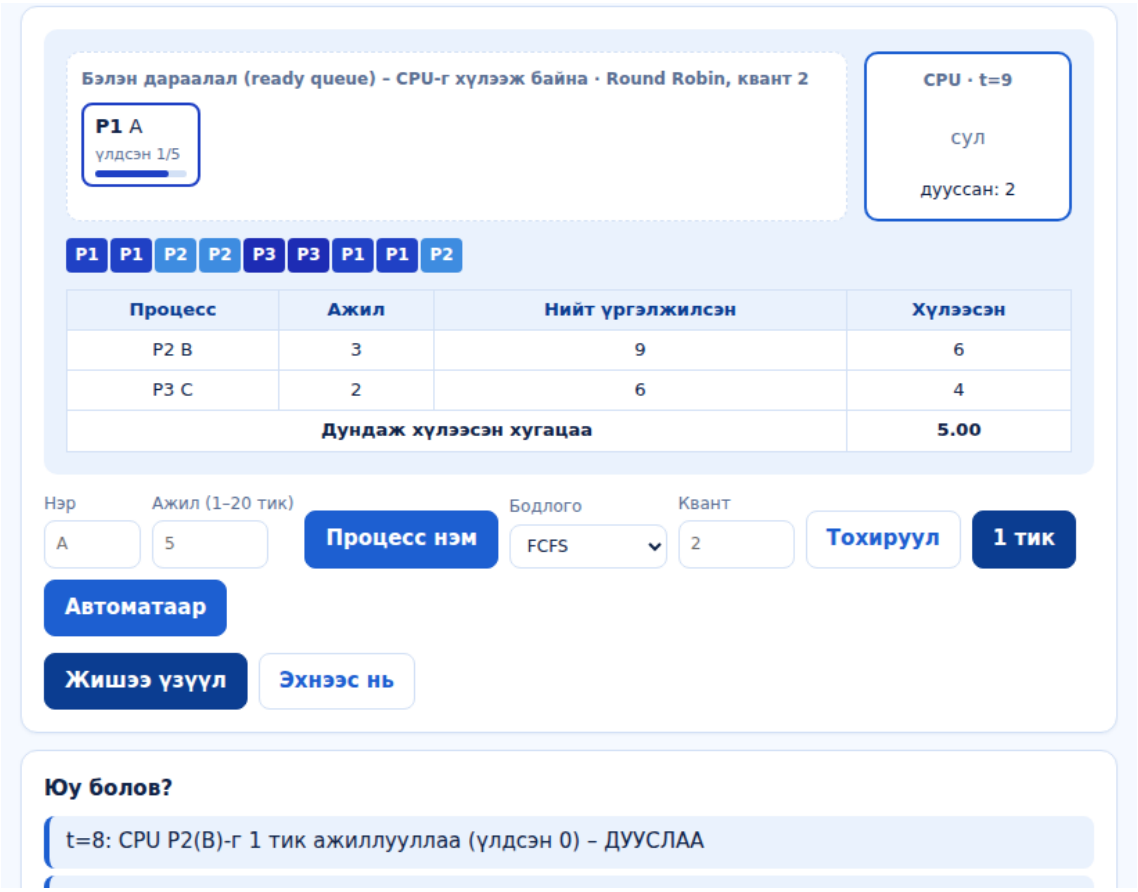
1.9-ийн дутууг нөхсөн хэсэг. (7) **Хайх үйлдэл** бүх бүтцэд: list find (индекс), queue/stack find (байрлал), tree find, bitmap test, hash get (алхамын тоотой). (8) **Hash map:** 1–16 bucket, хэш функц = тэмдэгтүүдийн кодын нийлбэр mod n, collision-ийг bucket доторх жагсаалтаар (chaining) шийднэ, 24 элементээр хязгаарлав. (9) **Тэнцвэржсэн мод:** tree mode avl – эргүүлэлт (rotr, rotl)-тэй AVL; төрлийг зөвхөн хоосон модонд солино. Энгийн мод 1–7-г дараалан оруулахад өндөр 7, AVL-д 3.

6. Туршилт ба UI дүрслэл

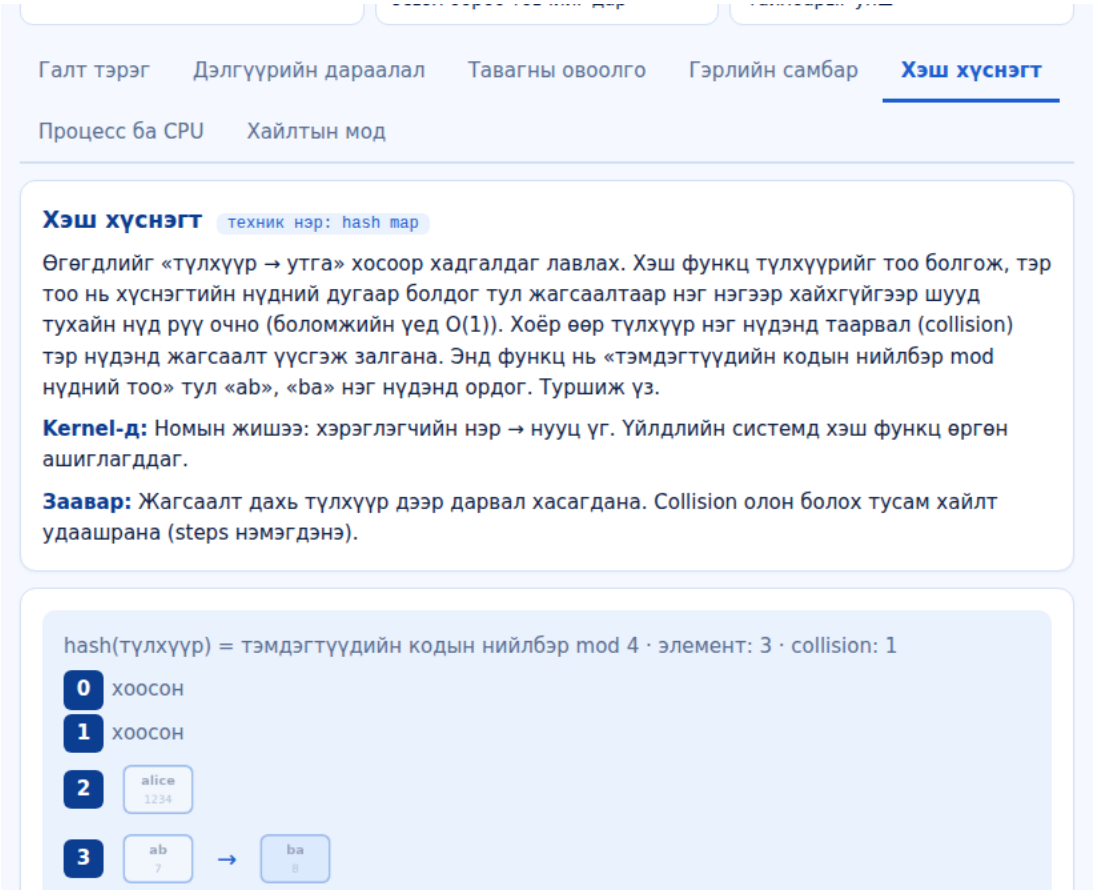
UI-д 7 tab бий: галт тэрэг (list_head: next/prev сум), дараалал, овоолго, гэрлийн самбар (bitmap, bit бүр нэг нүд), хэш хүснэгт (bucket бүрийн жагсаалт, collision), процесс ба CPU (бэлэн дараалал, Gantt), хайлтын мод (энгийн ба AVL, өндрийн заалттай). Бүтэц бүр дээр нэмэх, устгах, хайх товч байна. Хэрэглэгч огноо, тоо, bitmap-ийн хэмжээг өөрчилж, товч эсвэл командаар үйлдэл хийнэ; цайвар цэнхэр дүүргэлт сүүлийн өөрчлөлтийг, «Юу болов?» хэсэг OK (цэнхэр зураас) ба алдаа (улаан зураас, «Алдаа:»)-г харуулна. Reset ба script ачаалах боломжтой.



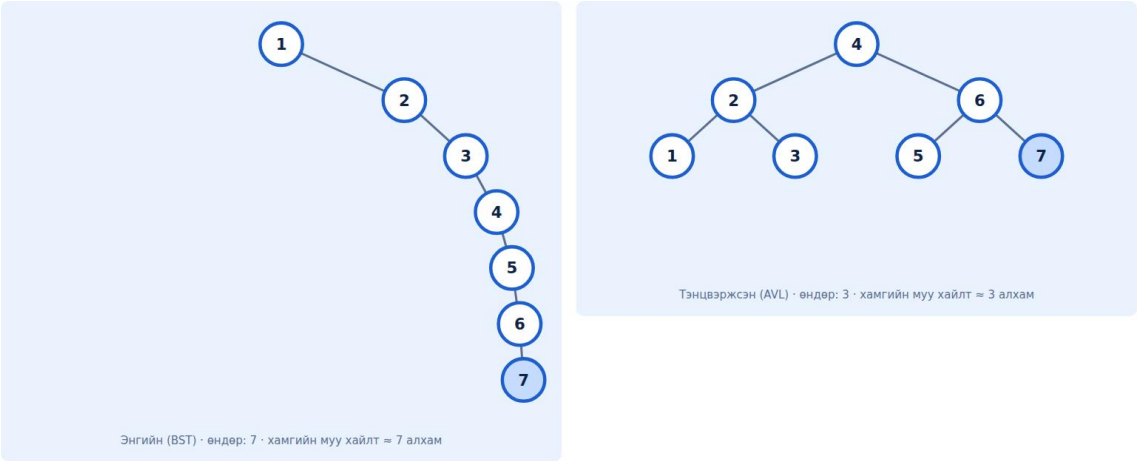
Зураг 1 – Галт тэрэг (list_head) tab: 17/3/1998 вагон салгагдаж, буруу огноо (31/4) «Юу болов?» хэсэгт тайлбартай татгалзагдсан.



Зураг 2 – Процесс ба CPU tab: Round Robin (квант 2), Gantt диаграм, процесс бүрийн хүлээсэн хугацаа.



Зураг 3 – Хэш хүснэгт (4 bucket): «bob», «ab», «ba» нэг 3-р bucket-д орж collision үүссэн; «ba»-г хайхад 3 алхам шаардлаа.



Зураг 4 – Эрэмбэлсэн 1–7-г оруулсны дараа: зүүн талд энгийн мод (өндөр 7, нэг шугам), баруун талд тэнцвэржсэн AVL мод (өндөр 3).

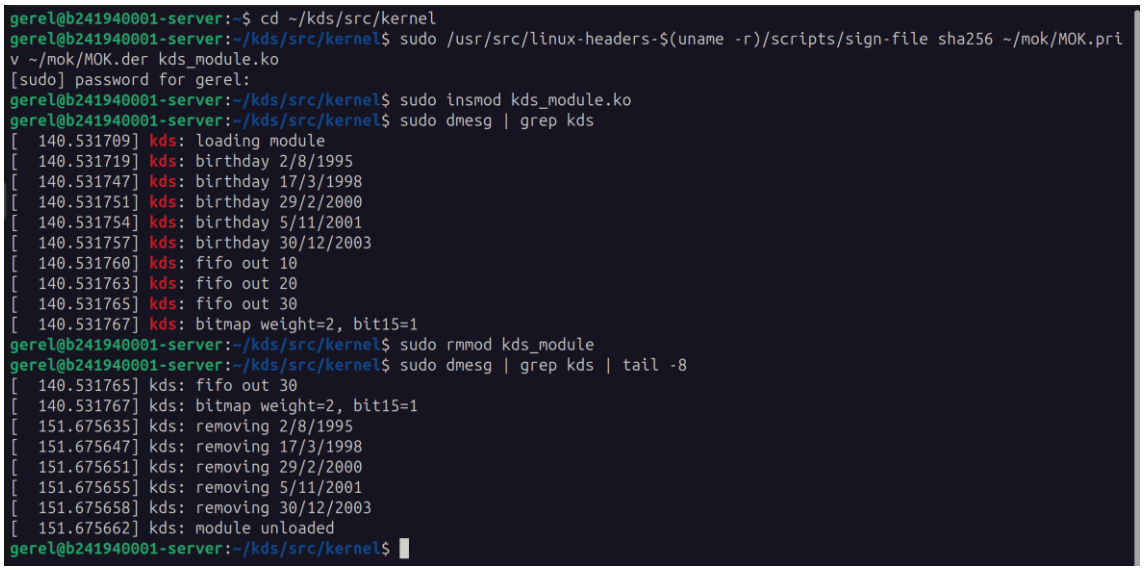
7. Туршилтын арга

Орчин: Ubuntu 24 (sandbox), gcc, node, Chromium (playwright). Тест: (a) examples/textbook.txt – сурах бичгийн дүр; (b) examples/all_structures.txt – бусад бүтэц; (c) examples/errors.txt – захын ба буруу оролт; (c2) examples/search.txt – 1.9-ийн хайлт, хэш collision, энгийн ба тэнцвэржсэн мод; (c3) tests/avl_check.py – 30 санамсаргүй гүйлтэд AVL-ийн өндрийн хязгаар ($\leq 1.44 \cdot \lg(n+2)$) ба in-order агуулга; (d) 5000 push/pop stress – ERR гарахгүй, ASan leak/UB мэдээлэхгүй; (e) UI-ийн хөдөлгүүрийг node-оор ажиллуулж C-ийн гаралттай diff-лэх; (f) **fuzz тест** – tests/fuzz_compare.py нь санамсаргүй командыг (буруу огноо, хэт том индекс, 8-аас олон процесс гэх мэт) C ба UI-д өгч гаралтыг мөр мөрөөр харьцуулна; (g) scheduler-ийн хэвлэлийг гараар тооцсоноор шалгасан (доорх хүснэгт); (h) kernel module-ийг linux-headers-7.0.0-38-generic-ээр compile хийх. (i) өөрийн машин (Ubuntu 24.04, kernel 6.11.0-17-generic) дээр build хийж, Secure Boot-ын улмаас гарын үсэг зурж, insmod ба rmmod хийж dmesg-ээр шалгах. Хүлээгдэх үр дүн нь tests/expected_*.out-д хадгалагдсан.

8. Үр дүн

Тест	Үр дүн
textbook	PASS
all_structures	PASS
errors	PASS
scheduler (RR, FCFS, алдаа)	PASS
scheduler_full (8 процессын хязгаар)	PASS
search (1.9: хайлт, hash, AVL)	PASS
avl_check (30 санамсаргүй гүйлт)	PASS
stress (5000×4 үйлдэл)	PASS, ERR 0
UI == C (6 script)	PASS ×6

Тест	Үр дүн
fuzz C == UI (8 seed × 5000 команд)	PASS ×8
Kernel module compile	PASS: gcc-13, linux-headers-7.0.0-38-generic ба өөрийн машины 6.11.0-17-generic, алдаагүй
Kernel module insmod / rmmod	PASS: Ubuntu 24.04, kernel 6.11.0-17-generic, dmesg гаралт Зураг 5-д



Зураг 5 – Kernel module-ийн бодит ажиллагаа

1.9-ийн хайлт, хэш хүснэгт, мод (examples/search.txt, холбогдох мөрүүд):

```
OK list find 17/3/1998 = index 1
OK list find 1/1/2001 = not found
OK queue find 20 = position 1
OK queue find 99 = not found
OK stack find 1 = position 2
OK stack find 7 = not found
OK hash size 4
OK hash put alice -> 1234 (bucket=2, size=1)
OK hash put bob -> 5678 (bucket=3, size=2)
OK hash put carol -> 9 (bucket=1, size=3)
OK hash put dave -> 11 (bucket=0, size=4)
OK hash put ab -> 7 (bucket=3, size=5, collision)
OK hash put ba -> 8 (bucket=3, size=6, collision)
HASH buckets=4 size=6 collisions=2: b0=[dave:11] b1=[carol:9] b2=[alice:1234] b3=[bob:5678 -> ab:7 -> ba:8]
OK hash get ba = 8 (bucket=3, steps=3)
OK hash get bob = 5678 (bucket=3, steps=1)
OK hash get zed = not found (bucket=3)
OK hash put bob = 0 (updated, bucket=3)
OK hash del alice (size=5)
ERR hash: key not found
HASH buckets=4 size=5 collisions=2: b0=[dave:11] b1=[carol:9] b3=[bob:0 -> ab:7 -> ba:8]
ERR hash: key and value must be 1..12 letters, digits or _
TREE size=5 height=5 inorder: 1, 2, 3, 4, 5
OK tree find 5 = yes
TREE(avl) size=7 height=3 inorder: 1, 2, 3, 4, 5, 6, 7
OK tree find 5 = yes
TREE(avl) size=6 height=3 inorder: 1, 2, 3, 5, 6, 7
```

Сурах бичгийн жишээний бодит гаралт:

```
OK list add_tail 2/8/1995 (size=1)
OK list add_tail 17/3/1998 (size=2)
OK list add_tail 29/2/2000 (size=3)
OK list add_tail 5/11/2001 (size=4)
OK list add_tail 30/12/2003 (size=5)
LIST size=5: 2/8/1995 -> 17/3/1998 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
```

```
OK list del 17/3/1998 (size=4)
LIST size=4: 2/8/1995 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 2/8/1995 (size=3)
OK list del 29/2/2000 (size=2)
OK list del 5/11/2001 (size=1)
OK list del 30/12/2003 (size=0)
LIST size=0: (empty)
```

Scheduler-ийн гараар шалгалт (Round Robin, квант 2; A=5, B=3, C=2 тик, бүгд t=0-д ирсэн):

Процесс	Дууссан t	Нийт үргэлжилсэн	Ажил	Хүлээсэн
C	6	6	2	4
B	9	9	3	6
A	10	10	5	5
Дундаж				5.00

Ижил ажил FCFS-д: A=0, B=5, C=8 хүлээнэ → дундаж 4.33. Round Robin нь богино процессуудыг түргэн хариулдаг боловч дундаж хүлээлт энд FCFS-ээс муу гарсан нь «квант, шилжилт»-ийн үнэ юм. Хэрэгжилтийн гаралт (examples/scheduler.txt):

```
OK sched policy rr q=2
OK proc new P1 A burst=5 (alive=1)
OK proc new P2 B burst=3 (alive=2)
OK proc new P3 C burst=2 (alive=3)
SCHED policy=rr(q=2) time=0 running=none ready=P1(A:5), P2(B:3), P3(C:2) done=0
TICK t=0 P1(A) remaining=4
TICK t=1 P1(A) remaining=3 preempted
TICK t=2 P2(B) remaining=2
TICK t=3 P2(B) remaining=1 preempted
TICK t=4 P3(C) remaining=1
TICK t=5 P3(C) remaining=0 finished
TICK t=6 P1(A) remaining=2
TICK t=7 P1(A) remaining=1 preempted
TICK t=8 P2(B) remaining=0 finished
TICK t=9 P1(A) remaining=0 finished
TICK t=10 idle
SCHED policy=rr(q=2) time=11 running=none ready=(empty) done=3
```

Буруу оролтын бодит гаралт (examples/errors.txt):

```
ERR list: day out of range for month
ERR list: day out of range for month
ERR list: month must be 1..12
ERR list: year must be 1900..2100
ERR list: usage <op> <day> <month> <year>
ERR list: 1/1/2000 not found
LIST size=0: (empty)
ERR list: missing operation
ERR unknown structure 'frob'
ERR queue: underflow (empty)
ERR stack: underflow (empty)
ERR queue: usage push <int>
ERR bitmap: size must be 1..128
ERR bitmap: size must be 1..128
ERR bitmap: index 99 out of range 0..31
ERR bitmap: index -1 out of range 0..31
OK tree insert 5 (size=1)
ERR tree: duplicate 5
ERR tree: 9 not found
```

9. Хэлэлцүүлэг

Бүх буруу оролт тодорхой мессеж өгч програм унасангүй. Хязгаарлалт: (1) kernel module-ийг зөвхөн нэг машин (Ubuntu 24.04, kernel 6.11.0-17-generic) дээр ачаалж туршсан; өөр kernel хувилбарт (API өөрчлөлт) дахин build хийх шаардлагатай байж болно. Secure Boot идэвхтэй орчинд гарын үсэггүй модуль ачаалагдахгүй (insmod: Key was rejected by service) тул MOK түлхүүрээр гарын үсэг зурсан; (1а) 1.9-д дурдсан red-black мод (rbtree.h), тусдаа singly/circular жагсаалтын хувилбар хэрэгжээгүй: жагсаалт нь kernel-ийн list_head шиг давхар холбоост дугуй, тэнцвэржсэн модыг AVL-аар загварчилсан; hash map нь тогтмол bucket тоотой (динамик resize байхгүй, хэш функц нь энгийн нийлбэр тул collision олон); (1б) scheduler нь бүх процесс t=0-д ирдэг гэж үздэг, I/O хүлээлт, тэргүүлэх эрэмбэ байхгүй (энгийн загвар); (2) UI болон C нь тусдаа хэрэгжилт (дахин бичсэн), ижил байдлыг тестээр баталсан боловч нэг эх кодоос үүсээгүй; (3) модыг тэнцвэржүүлээгүй; (4) queue/stack-ийн багтаамж тогтмол 8; (5) GitHub дахь commit түүхийг гишүүд өөрсдөө үүсгэнэ.

10. Дүгнэлт

Номын 1.9-ийн бүтэц бүрийг – холбоост жагсаалт, дараалал, стек, binary search tree ба тэнцвэржсэн мод, хэш хүснэгт, bitmap – хэрэгжүүлж, нэмэх, устгах, хайх үйлдлийн дараах төлөвийг ижил командын хэлээр CLI ба график UI-аас харуулж чадсан. Энгийн мод эрэмбэлсэн оролтод нэг шугам болдог (өндөр 7), тэнцвэржсэн мод өндрөө 3 байлгадаг гэдгийг, мөн хэш хүснэгтэд collision үүсэхэд хайлтын алхам нэмэгддэгийг UI дээр нүдээр харуулсан. Зөв ажиллагааг 6 скрипт, stress тест, AVL-ийн инвариант шалгалт, UI==C харьцуулалт (40 000 санамсаргүй команд), мөн kernel module-ийг жинхэнэ Linux (6.11.0-17) дээр ачаалж нотолсон. Нэмэлтээр «Processes» сэдэвтэй PCB ба ready queue-гээр FCFS, Round Robin-ийг хэмжигдэхүйц (хүлээлтийн хугацаа) харьцуулсан.

Ашигласан материал

Silberschatz, Galvin, Gagne – Operating System Concepts (Programming Projects); Linux kernel: include/linux/list.h, kfifo.h, bitmap.h баримт; AI хэрэгсэл: Claude (Anthropic) – тайлангийн ноорог бэлтгэхэд ашигласан.